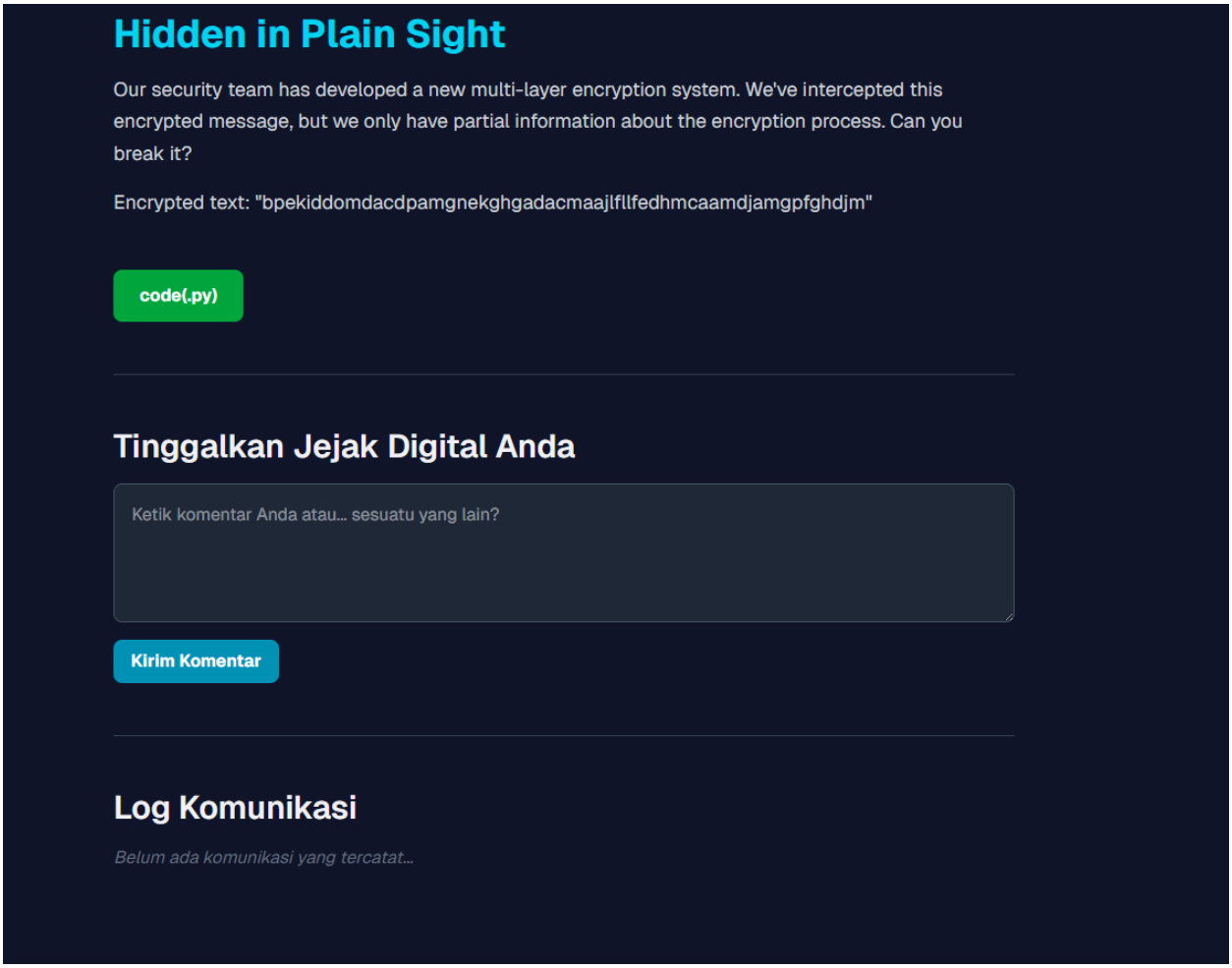
Hidden in Plain Sight

pangeran keputih

Web Exploitation

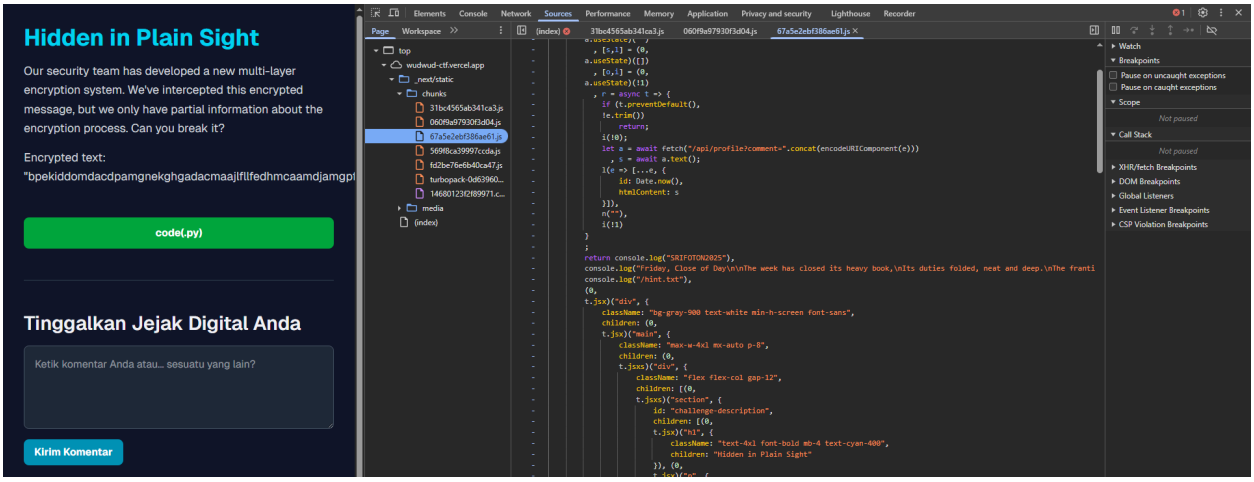
Write-up Penyelesaian

1.



Di awal, kita dikasih teks terenkripsi seperti diatas dan sebuah code.py. Berhubung nama dari challenge nya Hidden in Plain Sight, jadi kita coba inspect webnya

2.

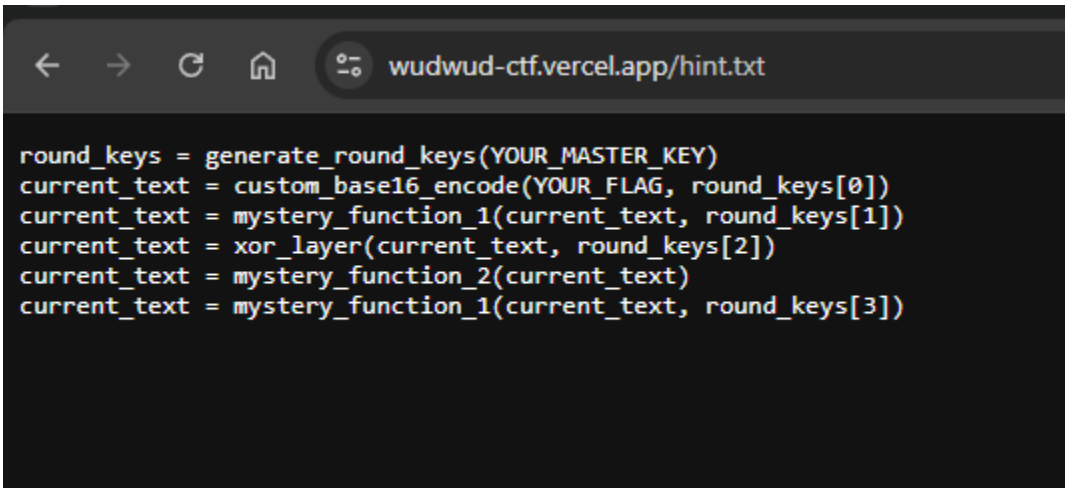


Setelah mencari-cari ternyata, ada salah satu javascript yang menarik perhatian saya.

```
}
;
return console.log("SRIFOTON2025"),
console.log("Friday, Close of Day\n\nThe week has c
console.log("/hint.txt"),
(0,
t.jsx)("div", {
    className: "bg-gray-900 text-white min-h-screen
```

Disana terdapat kalimat2/kata2 yang harusnya jadi hint.

3. Saya langsung coba untuk menuju ke page /hint.txt dan ternyata ada salah satu petunjuk untuk solve script nya

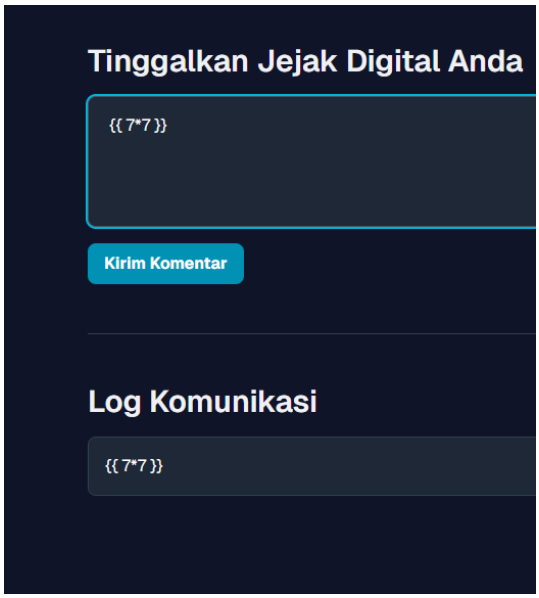


Dengan begitu kita tahu bahwa untuk mensolve soal ini, diperlukan sebuah MASTER_KEY Yang nantinya akan nge generate 4 round key.

Dan nanti urutan script dekripsi nya akan dibalik yaitu Inverse mystery_function_1 pakai round key ke-4, Inverse mystery_function_2 (fungsi ini self-inverse), Inverse xor_layer pakai round key ke-2, Inverse mystery_function_1 pakai round key ke-2, Decode custom_base16_encode pakai round key ke-0.

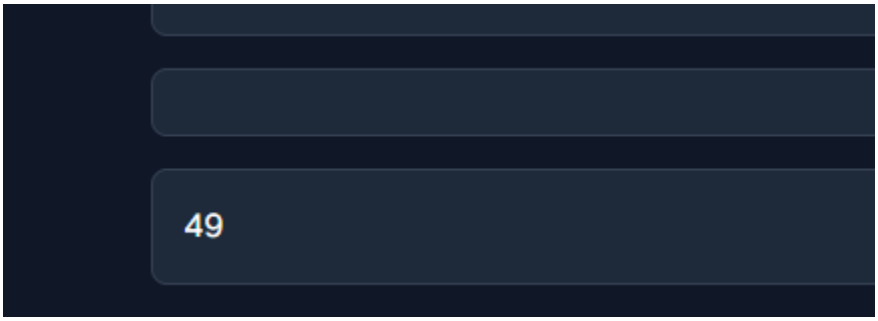
4.

Dari hint yang diberikan, yaitu no 1. Maka dugaan pertama saya adalah SSTI.
Untuk test, saya coba payload `{{ 7*7 }}`

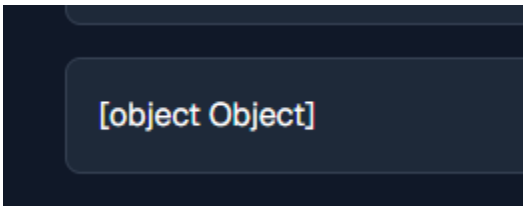


Dan ternyata tidak berhasil payloadnya
5. Lalu saya coba ganti dengan payload dengan <%= 7*7 %>

Dan ternyata ter eksekusi oleh web tersebut



6. Dengan begitu kita bisa mencoba untuk melihat environment variable dengan payload <%= process.env %> karena kemungkinan pakai EJS.



Dan ternyata outputnya menunjukan bahwa memang process.env itu merupakan sebuah object di server dan sayangnya di sini server hanya nunjukin tipe data nya aja. Jadi perlu kita ubah dalam bentuk string

7. Cara mengubah notasi nya menjadi sebuah string dengan JSON.stringify() yang nanti akan mengubah object process.env menjadi teks .JSON dengan payload

<%= JSON.stringify(process.env) %>

Dan output yang diharapkan muncul, dan terlihat bahwa master_key nya adalah comero_ctf_2025

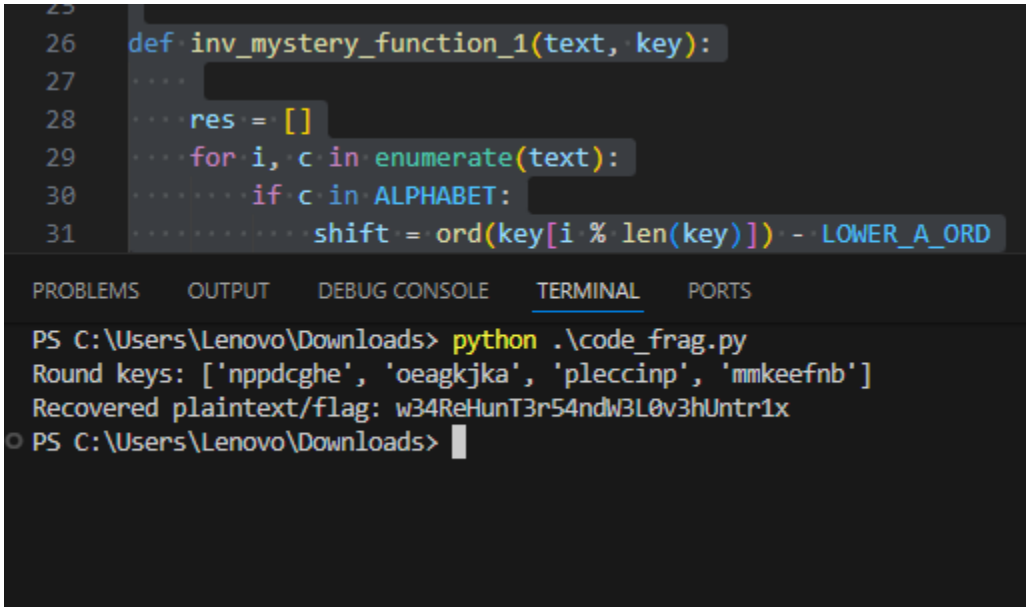
8. Langsung saja kita masukkan MASTER_KEY kedalam script yang telah kita buat

```python
#!/usr/bin/env python3

import argparse
import hashlib
import string
import sys

ALPHABET = string.ascii_lowercase[:16]  # 'abcdefghijklmnop'
LOWER_A_ORD = ord('a')

def generate_round_keys(master_key, rounds=4):

    keys = []
    current = master_key
    for i in range(rounds):
        h = hashlib.sha256(current.encode() +
str(i).encode()).hexdigest()
        round_key = ""
        # take bytes of the hash, map each byte to 0-15 via %16, pick
letter
        for j in range(0, 32, 2):
            hex_pair = h[j:j+2]
            idx = int(hex_pair, 16) % 16
            round_key += ALPHABET[idx]
        keys.append(round_key[:8])  # keys used are length 8 in the
recovered trace
    return keys

def inv_mystery_function_1(text, key):

    res = []
    for i, c in enumerate(text):
        if c in ALPHABET:
            shift = ord(key[i % len(key)]) - LOWER_A_ORD
            new_pos = ALPHABET.index(c)
            old_pos = (new_pos - shift) % 16
            res.append(ALPHABET[old_pos])
        else:
            # preserve characters outside ALPHABET unchanged
```

```
(defensive)
            res.append(c)
    return "".join(res)

def mystery_function_2(text, block_size=8):

    if len(text) % block_size != 0:
        text = text + 'a' * (block_size - (len(text) % block_size))
    out = []
    for i in range(0, len(text), block_size):
        block = text[i:i+block_size]
        # swap adjacent pairs
        for j in range(0, len(block), 2):
            if j+1 < len(block):
                out.append(block[j+1])
                out.append(block[j])
            else:
                out.append(block[j])
    return "".join(out)

def inv_xor_layer(text, key):

    res = []
    for i, out_c in enumerate(text):
        found = None
        key_char = key[i % len(key)]
        for possible_in in ALPHABET:
            xor_val = ord(possible_in) ^ ord(key_char)
            mapped = ALPHABET[xor_val % 16]
            if mapped == out_c:
                found = possible_in
                break
        if found is None:
            raise ValueError(f"No preimage found for position {i}
char {out_c}")
        res.append(found)
    return "".join(res)

def inv_custom_base16_encode(enc_text, key):
```

```python
    if len(enc_text) % 2 != 0:
        raise ValueError("encoded base16 length must be even")
    out_bytes = []
    for i in range(0, len(enc_text), 2):
        pair_idx = i // 2
        kchar = key[pair_idx % len(key)]
        rotation = (ord(kchar) - LOWER_A_ORD) % 16
        rotated = ALPHABET[rotation:] + ALPHABET[:rotation]
        c1 = enc_text[i]
        c2 = enc_text[i+1]
        if c1 not in rotated or c2 not in rotated:
            raise ValueError("encoded chars not in rotated alphabet")
        high = rotated.index(c1)
        low = rotated.index(c2)
        byte = (high << 4) | low
        out_bytes.append(byte)
    # Convert bytes to string (assuming printable ASCII flag)
    try:
        return bytes(out_bytes).decode('utf-8', errors='strict')
    except Exception:
        # fallback: return raw bytes as a latin-1 mapped string
        return bytes(out_bytes).decode('latin-1')

def decrypt(ciphertext, master_key):
    # generate round keys
    round_keys = generate_round_keys(master_key)
    # reverse order of encryption
    s0 = ciphertext
    s1 = inv_mystery_function_1(s0, round_keys[3])   # inverse of
last mystery_function_1
    s2 = mystery_function_2(s1)                      # self-inverse
    s3 = inv_xor_layer(s2, round_keys[2])            # inverse
xor_layer
    s4 = inv_mystery_function_1(s3, round_keys[1])   # inverse
mystery_function_1 with round_keys[1]
    flag = inv_custom_base16_encode(s4, round_keys[0])  # inverse
custom base16 encode
    return round_keys, flag
```

```python
def main():
    p = argparse.ArgumentParser(description="Decrypt custom CTF
ciphertext")
    p.add_argument("--cipher", "-c",
default="bpekiddomdacdpamgnekghgadacmaajlfllfedhmcaamdjamgpfghdjm",
                   help="ciphertext (default: the provided sample)")
    p.add_argument("--master", "-k", default="comero_ctf_2025",
                   help="master key (default: comero_ctf_2025)")
    args = p.parse_args()

    try:
        keys, flag = decrypt(args.cipher, args.master)
    except Exception as e:
        print("Decryption failed:", e, file=sys.stderr)
        sys.exit(2)

    print("Round keys:", keys)
    print("Recovered plaintext/flag:", flag)

if __name__ == "__main__":
    main()
```

9. 

Ketika kita run scriptnya, kita mendapat flagnya. Yaitu SRIFOTON{w34ReHunT3r54ndW3L0v3hUntr1x}